

Unity ゲーム開発における Web 可視化のための設計情報 JSON 抽出システムの実装と評価

Implementation and Evaluation of a Design Information JSON Extraction System for Web Visualization in Unity Game Development

福本陽翔¹
Haruto Fukumoto

田中康一郎¹
Koichiro Tanaka

九州産業大学¹
Kyushu Sangyo University.

1 まえがき

Unity を用いたゲーム開発では、プロジェクト規模の増大に伴い、スクリプト、Prefab、Scene などのアセット間の依存関係が複雑化する。本研究では、Unity プロジェクト内のアセット構造、スクリプト依存関係、Prefab 構造および Scene 構造を解析し、Web 可視化に利用可能な Node/Edge 形式の JSON として出力するシステムを実装、評価する。

2 提案手法

提案システムは Unity Editor 上で動作し、プロジェクト内のフォルダ、アセット、スクリプト、Prefab および Scene を解析対象とする。取得した情報は Node および Edge として整理し、JSON 形式で出力する。スクリプト解析では、クラス名、継承関係、インタフェース実装関係に加え、SerializeField 属性、GetComponent、Instantiate などの利用状況を解析し、スクリプト間の依存関係を抽出する。また、Prefab および Scene 解析では GameObject 階層と Component 構成を取得し、設計情報として記録する。さらに、同一フォルダ内のアセット群をモジュールとして扱い、プロジェクト構造を把握しやすい形式で整理する。

3 実装

本システムは C# を用いて実装し、Unity の AssetDatabase を利用してプロジェクト内の情報を取得した [1]。フォルダおよびアセット解析では、AssetDatabase.FindAssets により GUID、ファイルパス、アセット種別を取得し、フォルダとアセットの包含関係を Edge として記録した。スクリプト解析では、MonoScript クラスを用いてクラス情報を取得した [2]。さらに、ソースコードをテキスト解析することで、SerializeField 属性による参照関係や GetComponent、Instantiate などの利用関係を抽出した。Prefab および Scene 解析では、GameObject の階層構造と Component 構成を取得し、Node および Edge として記録した。解析方式として、Full Scan、Lightweight Scan、Lazy Scene Scan、Differential Scan の 4 方式を実装した。Full Scan は全情報を解析する基準方式である。Lightweight Scan は Scene および Prefab 内部構造の解析を省略する方式である。Lazy Scene Scan は Scene 内部構造の解析を遅延させる方式である。Differential Scan は前回解析結果をキャッ

シュとして利用し、変更がない場合に解析結果を再利用する方式である。

4 評価

規模の異なる 3 つの Unity プロジェクトを対象に評価を行った。評価対象は、Small、Medium、Large の 3 種類とし、それぞれアセット数とスクリプト数は、18/5、1386/101、3073/313 である。評価では、Full Scan を基準として、各解析方式の実行時間および Graph Completeness (以下、GC) を比較した。GC は、Full Scan で得られた Node 数および Edge 数に対する割合として算出した。

表 1 解析方式ごとの実行時間と GC

Project	実行時間 [ms]				GC [%]		
	Full	Light	Lazy	Diff	Light	Lazy	Diff
Small	837	46	205	31	61.3	61.3	100.0
Medium	2911	184	321	116	64.0	85.2	100.0
Large	26485	5683	1518	24564	5.9	40.1	100.0

表 1 より、Small および Medium では Differential Scan が最も高速であった。Small では Full Scan の 837 ms に対して 31 ms、Medium では 2911 ms に対して 116 ms であり、いずれも約 96% の解析時間短縮を確認した。一方、Large では Full Scan が 26485 ms、Differential Scan が 24564 ms であり、大きな短縮効果は得られなかった。これは、変更アセット検出時に全体解析へ戻る現在の実装に起因すると考えられる。Lightweight Scan は高速であるが、Prefab および Scene 内部構造を省略するため、GC が低下した。特に Large では、Full Scan で 33226 個の Node と 150706 個の Edge を抽出したのに対し、Lightweight Scan では 3051 個の Node と 3809 個の Edge にとどまり、GC は 5.9% であった。この結果から、大規模プロジェクトでは Prefab および Scene 内部構造が設計情報の多くを占めると考えられる。また、Lazy Scene Scan は Medium において 321 ms で 85.2% の GC を維持しており、概要把握に有効であることを確認した。

5 まとめ

評価の結果、Full Scan は完全な設計情報を取得できる一方で解析時間が増加し、Lightweight Scan は高速であるが情報量が減少することを確認した。また、Lazy Scene Scan は速度と情報量のバランスに優れ、Differential Scan は変更がない場合に完全性を維持したまま解析

時間を短縮できた。今後は、変更アセットのみを部分的に再解析する方式を導入するとともに、出力した JSON を Web 上で可視化するシステムへ発展させる。

参考文献

- [1] Unity Technologies. Assetdatabase, 2018.
- [2] Unity Technologies. Monoscript, 2020.